

Introduction

Array problems that ask you to "remove elements from the front or back" show up constantly in coding interviews, and they're deceptively tricky. Most people's first instinct is to simulate the process step by step — track the current min, remove it, check if it's at an end, repeat. That instinct is exactly what LeetCode 2091, Removing Minimum and Maximum From Array, is designed to punish.

This problem is a great teaching example because it tests something interviewers care about deeply: can you step back from a simulation and notice that most of the state you're tracking is irrelevant? Once you see that only two positions in the entire array matter, the problem collapses from "write a loop that mutates an array" into "compare three numbers." That shift in thinking — from simulation to direct computation — is one of the most valuable instincts you can build for interviews, and it's the whole point of this article.

Problem Overview

You're given an array of distinct integers. In one operation, you can either:

- Delete the leftmost element of the array, or
- Delete the rightmost element of the array

You need to find the minimum number of deletions required so that both the minimum value and the maximum value have been removed from the array. Order of deletions doesn't matter — only the count.

Since all values are distinct, there is exactly one minimum and exactly one maximum, so there's never a tie to break.

Example

Take the array:

```
[2, 10, 7, 5, 4, 1, 8, 6]
```

- The maximum, **10**, is at index **1**.
- The minimum, **1**, is at index **5**.

To remove both, you have three options:

1. **Delete from the front only**, far enough to pass index 5 (the min's position): that's 6 deletions.
2. **Delete from the back only**, far enough to pass index 1 from the right side: that's 7 deletions.
3. **Delete a bit from each end**: 2 deletions from the front (clears the max at index 1) and 3 deletions from the back (clears the min at index 5). Total: 5 deletions.

The answer is 5, because chopping from both ends lets you avoid dragging along all the elements between the min and max.

Intuition

The naive approach is to simulate: repeatedly look at the front and back of the array, check if either end holds the current min or max, delete it, and keep going until both are gone. It feels natural because the problem statement describes deletions happening one at a time.

But step back and ask: what actually determines the cost? Every element between the min and the max is never the target of a deletion decision — it just gets swept away as a side effect of clearing a path to the min or max. The only two facts that matter are:

- **Where is the minimum?** (its index, call it i)
- **Where is the maximum?** (its index, call it j)

Once you know these two positions, you don't need to simulate anything. You only need to compare three fixed ways of reaching both positions:

1. Clear everything from the front through whichever of i / j comes later.
2. Clear everything from the back through whichever of i / j comes earlier.
3. Clear from the front up through the earlier index, and from the back up through the later index.

Whichever of these three costs the least is your answer. No loop over the array's contents is needed beyond finding the two positions.

Brute Force Solution

Idea: Simulate the deletion process. Maintain pointers to the current front and back of the array (or a live min/max). At each step, check whether the front or back holds the current min or max, and delete accordingly, counting the number of operations until both have been removed.

Algorithm: 1. Track the current left and right boundary of the "remaining" array. 2. Repeatedly recompute (or track) the min and max within the remaining bounds. 3. Delete from whichever end gets you closer to removing the min or max. 4. Count operations until the min and max are both outside the remaining bounds.

Advantages: - Mirrors the problem description literally, so it's easy to explain conceptually. - Doesn't require any upfront insight about index positions.

Disadvantages: - Requires careful bookkeeping about which values have already been "seen" as deleted. - Naive re-scans for the current min/max at each step cost $O(n)$ per step, giving $O(n^2)$ overall. - Much harder to prove correct than directly comparing formulas, since greedy step-by-step choices can be subtle to justify.

Complexity: $O(n^2)$ time in the naive version, $O(1)$ extra space.

Optimal Solution

The optimal approach skips simulation entirely.

Step 1 — Locate the two positions that matter. Scan the array once to find the index of the minimum value and the index of the maximum value. Call the smaller of the two indices i and the larger j .

Step 2 — Compute three fixed costs.

Strategy	Cost	Meaning
Front-only	$j + 1$	Delete everything from index 0 through j
Back-only	$n - i$	Delete everything from index i through $n - 1$
Both-ends	$(i + 1) + (n - j)$	Delete $0 \dots i$ from the front, $j \dots n-1$ from the back

Step 3 — Return the minimum of the three.

That's the entire algorithm. There's no window to slide and no need to touch any element that isn't the min or max — the cost only depends on *positions*, not *values*.

```

Array:  [2, 10, 7, 5, 4, 1, 8, 6]
Index:  0  1  2  3  4  5  6  7
          ^max      ^min
          j=1       i=5

front-only  = j + 1          = 2
back-only   = n - i          = 3
both-ends   = (i+1) + (n-j)  = 6 + 2 = 8

```

Wait — walking through the numeric example from the transcript with $i=5$, $j=1$, $n=8$:

- front-only = $j + 1 = 2$ (this only clears the max, not the min — so front-only actually needs to go through $\max(i, j) = 5$, cost 6)
- back-only = $n - i = 3$ (this only clears the min, not the max, so back-only needs $n - \min(i, j) = n - 1 = 7$)
- both-ends = $(i + 1) + (n - j) \dots$

This is exactly why the swap step matters — the formulas above assume i is the min's index and j is the max's index *and* $i < j$. If the raw min/max scan gives you them in the opposite order, you must swap before applying the formulas, or the arithmetic silently answers a different question. With $i=1$ (earlier index) and $j=5$ (later index) after the swap:

- front-only = $j + 1 = 6$
- back-only = $n - i = 7$
- both-ends = $(i + 1) + (n - j) = 2 + 3 = 5$

The minimum is 5 — matching the walkthrough.

Python Code

```
def minimum_deletions(nums: list[int]) -> int:
    n = len(nums)

    i = nums.index(min(nums))
    j = nums.index(max(nums))

    # Ensure i is the earlier index and j is the later one.
    if i > j:
        i, j = j, i

    front_only = j + 1
    back_only = n - i
    both_ends = (i + 1) + (n - j)

    return min(front_only, back_only, both_ends)
```

Code Walkthrough

- $n = \text{len}(\text{nums})$: cache the array length, used in every formula.
- $i = \text{nums.index}(\min(\text{nums}))$: find the index of the smallest value. This is one linear pass.
- $j = \text{nums.index}(\max(\text{nums}))$: find the index of the largest value. Another linear pass.

- `if i > j: i, j = j, i` : since the formulas assume `i` comes before `j` in the array, swap them if the min happens to appear after the max.
- `front_only = j + 1` : the number of elements from index `0` up to and including `j`, the later of the two target indices.
- `back_only = n - i` : the number of elements from index `i` to the end of the array.
- `both_ends = (i + 1) + (n - j)` : delete the front chunk up to and including `i`, plus the back chunk from `j` to the end.
- `return min(...)` : pick whichever strategy costs the fewest deletions.

Dry Run

Array: `[2, 10, 7, 5, 4, 1, 8, 6]`, `n = 8`

1. `min(nums) = 1`, at index `5` → candidate `i = 5`
2. `max(nums) = 10`, at index `1` → candidate `j = 1`
3. Since `i (5) > j (1)`, swap: now `i = 1`, `j = 5`
4. `front_only = j + 1 = 6`
5. `back_only = n - i = 8 - 1 = 7`
6. `both_ends = (i + 1) + (n - j) = 2 + 3 = 5`
7. `min(6, 7, 5) = 5`

Return `5` — matching the manual trace from the intuition section.

Complexity Analysis

- **Time:** $O(n)$. Finding the min and its index, and the max and its index, are each a single linear scan. Everything after that is constant-time arithmetic.
- **Space:** $O(1)$. Only a handful of integer variables are used; no auxiliary array or extra structure is needed.

These bounds are correct because the algorithm never revisits array elements after the two initial scans — there's no nested loop, no recursion, and no growing data structure.

Alternative Solutions

A sliding window feels tempting because the problem involves cutting from both ends, but there's no window condition to satisfy — you're not searching for a contiguous range that meets some property, you're independently choosing a front cut and a back cut. Once you see that, the sliding window

framing dissolves and you're back to comparing three fixed formulas. There isn't a meaningfully different algorithmic approach beyond this — every correct solution reduces to locating `i` and `j` and comparing the same three costs, whether you compute the min/max with `min()` / `max()`, in a single combined loop, or via sorting (which would cost $O(n \log n)$ and is strictly worse).

Edge Cases

- **Two-element array:** `i` and `j` are `0` and `1` in some order; all three formulas correctly reduce to `2`.
- **Minimum and maximum adjacent:** the both-ends and front/back-only formulas still work without special-casing, since the swap step already normalizes order.
- **Minimum or maximum at an array boundary:** if the max is already at index `n - 1`, `back_only` naturally comes out cheap since `n - i` only pays for reaching the min.
- **Large arrays (up to 100,000 elements):** the $O(n)$ scan keeps this well within time limits; avoid any $O(n \log n)$ sorting-based shortcut.
- **All values distinct (guaranteed by constraints):** no need to handle duplicate min/max values, since the problem guarantees uniqueness.

Common Mistakes

1. **Forgetting to swap `i` and `j`.** If the min's raw index happens to be larger than the max's raw index, plugging them into the formulas unswapped produces wrong (often negative or oversized) costs.
2. **Simulating deletions with a loop.** This works but is unnecessarily slow and error-prone compared to the direct formula approach.
3. **Confusing which formula pairs with which strategy.** Mixing up `front_only` and `back_only` silently returns a valid-looking but incorrect number.
4. **Re-scanning the array inside a loop to recompute min/max.** This degrades an $O(n)$ solution into $O(n^2)$ without any correctness benefit.
5. **Off-by-one errors in the both-ends formula.** Forgetting the `+1` on `i` or miscounting `n - j` is the most common source of a wrong final answer.

Interview Questions

- How would you modify this solution if duplicate values were allowed?
- Can you solve this without using Python's built-in `min()` / `max()` / `index()`, in a single pass?

- What's the time complexity if you used `sorted(nums)` to find the min and max instead?
- How would the solution change if you could only delete from the front, never the back?
- Can you prove why simulating the deletions greedily is never actually necessary?

Similar Problems

- **LeetCode 1750 – Minimum Length of String After Deleting Similar Ends:** another two-ends deletion problem where the key insight is tracking pointers from both sides rather than simulating character-by-character removal.
- **LeetCode 42 – Trapping Rain Water:** also benefits from recognizing that only certain boundary values (max heights from each side) matter, not every element.
- **LeetCode 11 – Container With Most Water:** another "two ends matter, middle doesn't" problem solved with direct index comparison instead of brute force.
- **LeetCode 1855 – Maximum Distance Between a Pair of Values:** similarly reduces to comparing index relationships rather than exhaustively checking pairs.

Key Takeaways

The core lesson of LeetCode 2091 isn't about arrays at all — it's about resisting the urge to simulate a process when the problem can be reduced to a handful of positions and a direct formula. Find the two indices that actually matter, derive the fixed costs for reaching them, and compare. That's $O(n)$ time, $O(1)$ space, and a technique that generalizes to many "delete/trim from both ends" problems.

Watch the Video

For the full step-by-step derivation, including two quiz breaks to test your understanding before the answer is revealed, watch the video here: {LINK}

About the Series

This breakdown is part of **Fun with Learning Technology**, a daily series that takes one LeetCode problem and works through it from first principles — building intuition before writing a single line of code, and explaining *why* an approach works rather than just *what* it does. New problems are posted every day.

Call To Action

If this walkthrough helped the solution click, give the video a like on YouTube and subscribe so future daily problems land in your feed. For deeper write-ups like this one delivered straight to your inbox, subscribe to the Substack newsletter. Drop a comment with which strategy you guessed first, and share this with anyone prepping for coding interviews.

Quick Review

Two-pointer removal pattern: when to think 'shrink from both ends'?

When only min/max element positions matter, not the values between them — think removing from either end of an index range.

Time complexity of the min/max removal solution?

$O(n)$ — one pass to find min/max index, then constant-time comparisons of three removal strategies.

Space complexity of the min/max removal solution?

$O(1)$ — only tracks two index positions, no extra array or auxiliary structure needed.

Trickiest edge case: min and max are the same element?

When array has one element, $\text{minIdx} == \text{maxIdx}$ — one removal step clears both, don't double-count it.

Why does simulating actual removals feel right but fail here?

Physically deleting and reindexing costs $O(n)$ per removal; tracking left/right boundary counts avoids that entirely.

The three formulas: what do they each compute?

Removals via left-only, right-only, and both-ends-simultaneously; take $\min(\text{left} + \text{right}, \max(\text{left}, \text{right}))$ across all three.